SYSTEM PROGRAMING – TASK 1

Requirements:

1. **Implement a server-side application as a console program**

Implemented in main.py, which acts as the console entry point of the application. It starts the server, submits sample client requests, waits for processing, and then stops the server and prints statistics

The server logic is implemented in server.py

2. **Processes client requests**

Request class defined in models.py. It defines the structure of a client request

The requests are created in main.py and are submitted to the server, where they are processes.

In server.py, the request are processes with:

- worker_loop(): Each worker thread runs this loop, waiting for requests and processing them.
- process_request(): Call in worker_loop(). Each worker retrieves a request from the shared queue and handles it by checking the cache, computing the result if needed, and returning the response.

3. **Generates responses based on access to shared resources**

The response is obtained in server.py

```
state, data = self.cache.get_or_reserve(request.resource_key)
```

But it is generated in cache.py with the function get_or_reserve(self, key). These responses are bases on access to a shared resource (cache). It can be three different responses:

- Hit: If the entry is ready and valid
- Wait: If the entry is not ready, because another worker is computing it
- Compute (Miss): If the key is not in the cache or if the entry is ready but not valid (less time than TTL)

4. **Support concurrent handling of multiple requests (request reception and processing performed in parallel)**

This is implemented in server.py with:

- Submit_request(): where the requests are register and added to the shared queue.
- Start(): create multiple threads and allow multiple of them to execute at the same time. Each worker execute worker_loop() to process the requests concurrently.

5. **The architecture should be based on separation of concerns between request intake and processing**

Request intake: submit_request() in server.py. The server receives the request, registers it and add it to the queue.

Processing: worker_loop() and process_request()

In request_queue.py, the class RequestQueue is the intermediary between intake and processing. With the function put() implements the request intake and with the function get() the processing.

6. **Incoming requests are placed into a shared data structure**

Implemented in request_queue.py with the class RequestQueue. It is initialized with:

- _queue: stores requests
- _lock: protects against concurrent access
- _not_empty: allows blocking wait
- _closed

With the function put() the requests are added to the queue and notify the workers that there are requests. This function is called in server.py in the function submit_request()

7. **A pool of worker threads performs the processing**

In server.py with the function start(). It creates the number of workers defined and they are added to a list call 'workers'. Each one of them execute the function worker_loop()

Each thread is created only once and they are reused to execute the different threads.

8. **The number of concurrently active workers must be controlled**

This is implemented with a Semaphore (worker_slot) and the variable max_active_workers.

In the function worker_loop() if there are slots free on the Semaphore, the worker process the request. When it is finished, the slot is free and another worker can enter. If there are no slots free, the worker is blocked.

9. **Synchronization between threads must be blocking and implemented explicitly using constructs from the threading module, such as Lock, Condition, or equivalent mechanisms.**

Blocking synchronization: when the worker has to wait, the function wait() make the worker wait without consuming CPU

Explicit synchronization:

- threading.Lock(): used to protect the queue
- threading.Condition(self._lock): will be used to wait for the queue to become non-empty.
- threading.Semaphore(): if there are no free slots the worker is blocked

10. **The application must be executed in a free-threading (no-GIL) environment**

It does not rely on implicit thread-safety of Python data structures. Instead, it uses locks, conditions, and semaphores to ensure correct behavior under true parallel execution.

11. **All shared resources must be accessed in a thread-safe manner using explicit synchronization.**
12. **Implicit thread-safety of built-in data structures must not be assumed.**

The request queue is protected with a lock and a condition, the cache is protected with a lock plus per-entry conditions, and the server statistics are protected with a dedicated lock. This ensures correct behavior under concurrent access.

13. **The system must include an in-memory caching mechanism for previously computed results.**

Implemented in cache.py with the class ThreadSafeCache. Previously computed results are stored in a shared dictionary, so future requests for the same resource can reuse them instead of recomputing them.

14. **The cache must be thread-safe and support a defined management policy**

Protected with threading.Lock(). It avoids multiple threads modifying or reading cache at the same time.

Management policy: TTL. Defined in the Cache's Constructor and used on the function is_valid(). This means that a value in cache is only valid during a period of time. If the value is no valid anymore it is replaced and recalculated.

15. **If multiple threads request the same resource concurrently, the computation must be performed only once, while other threads wait for the result.**

Implemented in cache.py. The function get_or_reserve() returns the state:

- Hit: If the entry is ready and valid
- Wait: If the entry is not ready, because another worker is computing it
- Compute (Miss): If the key is not in the cache or if the entry is ready but not valid (less time than TTL)

Only the worker that received 'compute' calculate the result, store it and wake up the other workers.

16. **The application must log request handling and system behavior, including success, failure, and cache usage.**

Implemented in logger_setup.py. A logger is created so the messages are saved and can be used later to analyzed them.

The format is: [%(threadName)s] %(levelname)s: %(message)s

Even though is implemented in logger_setup.py, it is used in server.py self.logger.info()

17. **Logging must be thread-safe.**

Logging is thread-safe because the logging module handles synchronization internally. Also, all the threads use the same logger.

18. **The system should also ensure proper error handling and stable operation under concurrent load.**

Error handling: a try/except block is added in the process_request(), so exceptions are caught and logged instead of terminating the worker. Also, in worker_loop() a try/finally bloack assure that the Semaphore is always released.

Stable operation:

- There are no race conditions because everything (queue, cache and server statistics) is protected
- There is no busy waiting because the threads don't consume CPU due to wait() function
- Concurrency Control with a Semaphore

19. **Analyze the applied synchronization mechanisms, identify critical sections, and discuss system behavior under varying levels of concurrency.**

Synchronization mechanism:

- Lock: protect the queue, the cache, the server statics -> mutual exclusion
- Condition: used for blocking wait between threads -> blocking wait / coordination
- Semaphore: limit the number of workers active. -> concurrency control

Critical sections:

- Request_queue.py -> the functions put(), get() and close() because multiple threads may insert, remove, or inspect queue state concurrently
- Cache.py -> the functions get_or_reserve() and store() because multiple workers may query or update the same shared dictionary concurrently
- Cache.py -> the function wait_for_value() because waiting threads depend on that shared state
- Server.py -> statistics updates because several threads modify the same counters

SYSTEM BEHAVIOR

1. **Low concurrency ->** the system behaves almost sequentially, with minimal contention and no waiting between workers

num_workers=1
max_active_workers=1
cache_ttl=15

```
===== SERVER STATISTICS =====
Total requests received : 4
Total requests processed: 4
Cache hits              : 3
Cache misses            : 1
Cache waits             : 0
Computed results        : 1
=============================
```

The four requests arrived and all of them where processed (the system works correctly)
Only the first request 'missed' the cache and had to calculate the value. (cache misses = 1)
The other three requests used the cache instead of calculate the value (cache hits = 3).
There are no cache waits, which shows that no threads were executing concurrently on the same resource.
Only one computation is performed, which confirms that the cache coordination mechanism works correctly

2. **Average concurrency (same key)** -> only one thread computes the result while the others wait, which avoids duplicated work

```
===== SERVER STATISTICS =====
Total requests received : 4
Total requests processed: 4
Cache hits              : 2
Cache misses            : 1
Cache waits             : 1
Computed results        : 1
============================
```

num_workers=4
max_active_workers=2
cache_ttl=15

The four requests arrived and all of them where processed (the system works correctly)
Only the first request 'missed' the cache and had to calculate the value. (cache misses = 1)
One of the requests arrives while the computation is still in progress, resulting in a cache wait.
This shows that multiple threads are active at the same time.
The other two requests (because they arrive later) used the cache instead of calculate the value (cache hits = 2).
Only one computation is performed, which confirms that the cache coordination mechanism works correctly

3. **High concurrency (same key) ->** the cache coordination mechanism works correctly: one thread computes and all others block until the value becomes available

```
===== SERVER STATISTICS =====
Total requests received : 6
Total requests processed: 6
Cache hits              : 0
Cache misses            : 1
Cache waits             : 5
Computed results        : 1
============================
```

num_workers=6
max_active_workers=6
cache_ttl=15

The six requests arrived and all of them where processed (the system works correctly)
Only the first request 'missed' the cache and had to calculate the value. (cache misses = 1)
The other ones arrive when while request1 is calculating so all them have to wait (cache waits = 5). That's why there is no hit, because all of them arrive at the same time.
Only one computation is performed, which confirms that the cache coordination mechanism works correctly

4. **High concurrency (different key) ->** the system can exploit parallelism better because requests do not contend for the same cache entry

```
===== SERVER STATISTICS =====
Total requests received : 6
Total requests processed: 6
Cache hits              : 0
Cache misses            : 6
Cache waits             : 0
Computed results        : 6
============================
```

num_workers=6
max_active_workers=6
cache_ttl=15

The six requests arrived and all of them where processed (the system works correctly)
All the six requests missed cache because they are asking for different values, so any of them is calculated when they arrive. There is no wait because they arrive at the same time, but they are working with different keys.
Six computations are performed, which confirms that the cache coordination mechanism works correctly